A MAJOR PROJECT REPORT

on

AI-BASED PREDICTIVE MAINTENANCE PLATFORM

Machine Failure Prediction, Anomaly Detection and Maintenance Workflow using Machine Learning

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

[Branch / Department]

Submitted by

GAURAV KUMAR

Roll No.: [Roll Number]

Under the guidance of

[Guide Name, Designation]

[College / Institute Name]

[University Name]

Session: [20XX - 20XX]

CERTIFICATE

This is to certify that the major project report entitled **"AI-Based Predictive Maintenance Platform"** submitted by **Gaurav Kumar** (Roll No. [Roll Number]) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in [Branch / Department] at [College / Institute Name] is a record of the candidate's own work carried out under my supervision and guidance. The matter embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

[Guide Name]
Project Guide

Date: [Date]

[Head of Department]
Head of Department

DECLARATION

I, **Gaurav Kumar**, hereby declare that the project report entitled **"AI-Based Predictive Maintenance Platform"** is my own work, carried out under the guidance of [Guide Name]. The dataset used for development is synthetic and follows the structure of the publicly available AI4I 2020 Predictive Maintenance dataset; the real AI4I 2020 dataset was additionally used for comparison. All external sources, libraries and datasets used have been acknowledged in the references. This work has not been submitted to any other institution for any degree or diploma.

Gaurav Kumar
Roll No.: [Roll Number]
Date: [Date]

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, [Guide Name], for the valuable guidance, constant encouragement and constructive feedback throughout the development of this project. I am also thankful to the Head of the Department and all the faculty members of the [Branch / Department] for providing the facilities and an environment that made this work possible.

I acknowledge the open-source community behind Python, Pandas, NumPy, Scikit-learn, Flask, Matplotlib and Seaborn, whose tools made it possible to build the complete system, and the authors of the AI4I 2020 Predictive Maintenance dataset for making a realistic benchmark publicly available.

Finally, I thank my family and friends for their support and motivation during the course of this project.

Gaurav Kumar

ABSTRACT

Unplanned machine failure is one of the costliest problems in manufacturing, because it stops production, damages equipment and creates safety risks. Time-based maintenance replaces parts too early, while reactive maintenance repairs them too late. Predictive maintenance tries to find the middle path: it uses sensor data to estimate when a machine is likely to fail, so that maintenance can be planned just in time.

This project presents an AI-Based Predictive Maintenance Platform that goes beyond a stand-alone machine learning model. It consists of (i) a complete machine learning pipeline (data cleaning, leakage-safe feature engineering, time-based train/test split, comparison of four classifiers, evaluation with metrics suited to imbalanced data), (ii) additional models for anomaly detection, failure-type prediction and Remaining Useful Life (RUL) estimation, (iii) a simple, dependency-free explanation of why a machine is at risk, and (iv) a Flask web application with nine pages, a REST API, a SQLite database and a maintenance workflow that carries a prediction all the way to a completed maintenance record.

The dataset contains 10000 sensor readings from 50 machines with a failure rate of about 3%, so accuracy alone is misleading. The models are therefore compared using Precision, Recall, F1-score, ROC-AUC and PR-AUC on the most recent 25% of the timeline. The primary gradient-boosted model reached a precision of 0.908, recall of 0.887, F1-score of 0.897 and PR-AUC of 0.937 on the unseen test period, clearly outperforming Logistic Regression, a Decision Tree and a Random Forest in the balance between missed failures and false alarms. The same pipeline, run on the real AI4I 2020 dataset, produced an F1-score of 0.794 and ROC-AUC of 0.984. The system is covered by 46 automated tests.

Keywords: Predictive Maintenance, Machine Learning, Gradient Boosting, Class Imbalance, Anomaly Detection, Remaining Useful Life, Explainability, Flask.

TABLE OF CONTENTS

Chapter 1: Introduction

1.1 Background

Modern factories depend on rotating machines such as motors, pumps, spindles and compressors. When one of them fails without warning, the whole production line may stop. Industry studies commonly report that unplanned downtime costs manufacturers a very large amount of money every year, apart from the damage to equipment and the risk to workers.

There are three broad maintenance strategies. In reactive maintenance a machine is repaired after it breaks. In preventive (time-based) maintenance parts are replaced on a fixed schedule whether or not they need it. In predictive maintenance the actual condition of the machine, measured through sensors, is used to decide when maintenance is needed. Predictive maintenance reduces both unnecessary replacements and unexpected breakdowns, but it needs a reliable way to turn raw sensor values into a decision.

1.2 Problem Statement

Given periodic sensor readings from industrial machines (air temperature, process temperature, rotational speed, torque and tool wear), build a system that (a) estimates the probability that a machine will fail, (b) detects unusual operating behaviour, (c) suggests the most likely type of failure and a rough remaining useful life, (d) explains in simple terms why a machine is considered at risk, and (e) helps the maintenance team act on the prediction through a web dashboard and a maintenance record workflow.

1.3 Objectives

- To design a leakage-safe machine learning pipeline for highly imbalanced failure data.
- To compare baseline models (Logistic Regression, Decision Tree, Random Forest) with a gradient-boosted model using metrics suitable for imbalanced classification.
- To add anomaly detection (Isolation Forest), failure-type classification and an experimental RUL estimator.
- To provide a simple explanation of each high-risk prediction without relying on heavy external libraries.
- To build a Flask web application with a database-backed dashboard, REST API and maintenance workflow.
- To verify the system with automated tests and to report its limitations honestly.

1.4 Scope of the Project

The project covers the complete path from data to decision: data preparation, model training, evaluation, deployment in a web application, monitoring through prediction history and a simple drift indicator, and maintenance record management. It is a university-level prototype

built on a synthetic dataset that follows the AI4I 2020 structure. It does not connect to live sensors, and user authentication is intentionally left out of scope (see Chapter 9).

1.5 Motivation

Many student projects stop at "train a model and print the accuracy". Real maintenance teams, however, do not need an accuracy number; they need to know which machine to look at first, why, and what to do about it. This project was motivated by the idea of building the whole workflow around the model, so that a prediction ends in an action, and by the desire to apply sound machine learning practice (time-based validation, no leakage, correct metrics) instead of only chasing a high score.

1.6 Organisation of the Report

Chapter 2 reviews related work. Chapter 3 gives the system requirements. Chapter 4 describes the system design with architecture, data flow and database diagrams. Chapter 5 explains the dataset, feature engineering, models and the web application. Chapter 6 presents the results. Chapter 7 describes testing. Chapter 8 shows the user interface through screenshots. Chapter 9 lists limitations, and Chapter 10 concludes the report and gives the future scope.

Chapter 2: Literature Review

2.1 Predictive Maintenance with Machine Learning

Carvalho et al. (2019) reviewed machine learning methods applied to predictive maintenance and found that Random Forests, Support Vector Machines, neural networks and gradient-boosted trees are the most frequently used methods, while public, realistic data and clear evaluation protocols remain a common weakness. Run-to-failure benchmarks such as the NASA C-MAPSS turbofan data (Saxena et al., 2008) made it possible to compare RUL estimation methods, but they describe engines rather than general industrial machines.

2.2 The AI4I 2020 Dataset

Matzka (2020) published the AI4I 2020 Predictive Maintenance dataset, a synthetic dataset of 10,000 records that reflects real milling-machine failure modes. Each record has product type (L, M, H), air temperature, process temperature, rotational speed, torque and tool wear, plus a machine-failure label and five failure-mode flags: tool wear failure (TWF), heat dissipation failure (HDF), power failure (PWF), overstrain failure (OSF) and random failures (RNF). The dataset is widely used because it is small, clean and has realistic class imbalance. A limitation relevant to this project is that its rows are independent samples and do not form a time series per machine.

2.3 Tree Ensembles and Gradient Boosting

Breiman (2001) introduced Random Forests, which average many decorrelated decision trees and are strong baselines on tabular data. Gradient boosting builds trees sequentially so that each tree corrects the errors of the previous ones; XGBoost (Chen and Guestrin, 2016) is a widely used, regularised and efficient implementation. These models capture non-linear sensor interactions without feature scaling, which suits mechanical data where failure depends on combinations such as high torque together with high tool wear.

2.4 Imbalanced Data and Evaluation Metrics

When only about 3% of records are failures, a model that always predicts "healthy" is 97% accurate and useless. SMOTE (Chawla et al., 2002) generates synthetic minority samples, but interpolating between neighbours can create unrealistic points and mixes information across time in sequential data. Class weights are a simpler alternative that changes the loss, not the data, and this project uses them. Saito and Rehmsmeier (2015) showed that the precision-recall curve is more informative than the ROC curve on imbalanced data because it is not inflated by the large number of true negatives; hence PR-AUC is reported alongside ROC-AUC.

2.5 Anomaly Detection

Isolation Forest (Liu, Ting and Zhou, 2008) isolates observations by random splits; abnormal points need fewer splits and receive higher anomaly scores. It can be trained on healthy data only, which is attractive in maintenance because many failure modes are rare or unseen. This project uses it as a second signal next to the supervised classifier.

2.6 Explainability

SHAP (Lundberg and Lee, 2017) explains individual predictions by assigning each feature a contribution value. It is powerful but adds a dependency and extra computation. For a maintenance operator, a group-level explanation ("tool wear and torque are the main drivers") is often more useful than 17 separate feature values. This project therefore uses a lightweight what-if method: each sensor group is replaced by its typical (median) value and the drop in failure log-odds is measured.

2.7 Research Gap Addressed by this Project

Most published work and most student projects focus on the model. This project tries to cover the surrounding system as well: leakage-safe validation, an explicit think-through of imbalance, a single shared preprocessing code path for training and serving, an explanation layer, a database-backed dashboard and a maintenance workflow, together with an honest statement of limitations.

Chapter 3: System Requirements and Analysis

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	Accept a sensor reading (machine, temperatures, speed, torque, tool wear) and validate it.
FR2	Return failure probability, risk level (LOW / MEDIUM / HIGH) and predicted class.
FR3	Detect abnormal operating behaviour and show an anomaly level.
FR4	Suggest the most likely failure type and recommended action with priority.
FR5	Estimate remaining useful life (experimental).
FR6	Explain which sensor groups contribute most to a high-risk prediction.
FR7	Store readings, predictions and maintenance records in a database.
FR8	Show a dashboard, machine list, machine detail, analytics, anomalies, history and model performance pages.
FR9	Allow creating a maintenance record from a prediction and marking it completed.
FR10	Provide a JSON REST API for prediction, summaries, machines, history, maintenance and drift.

3.2 Non-Functional Requirements

- **Reproducibility:** fixed random seeds and a single command (python -m src.train) to rebuild all artefacts.
- **Consistency:** the same preprocessing functions are used for training and prediction, so no train/serve mismatch can occur.
- **Usability:** clear messages for invalid input, and a dashboard that is readable without ML knowledge.
- **Reliability:** friendly error pages, input range checks and HTML escaping of user text.
- **Portability:** runs on Windows, Linux and macOS with Python 3.10+ and no external service; charts are inline SVG, so no internet or JavaScript library is needed.

3.3 Hardware and Software Requirements

Table 3.2: Software and hardware requirements

Item	Specification
Processor / RAM	Any modern dual-core CPU, 4 GB RAM or more (training takes about 30 seconds)
Operating system	Windows 10/11, Linux or macOS
Language	Python 3.10 or newer
Libraries	pandas, NumPy, scikit-learn, XGBoost (optional), matplotlib, seaborn, Flask, joblib

Item	Specification
Database	SQLite (built into Python)
Browser	Any modern browser (Chrome, Edge, Firefox)

3.4 Users and Use Cases

Table 3.3: Main use cases

Use case	Actor	Description
Check machine health	Maintenance engineer	Enters the latest reading and views risk, likely failure, explanation and action.
Monitor plant	Supervisor	Uses the dashboard to see healthy / medium / high-risk machines and recent alerts.
Plan maintenance	Supervisor / technician	Creates a maintenance record with priority, technician and date, and later marks it completed.
Review model	Student / examiner	Opens the model performance page to see comparison tables and plots.
Integrate	External system	Calls the REST API to get predictions or summaries in JSON.

Chapter 4: System Design

4.1 Architecture

The system follows a layered design. The presentation layer is the web browser. The application layer is a Flask application that validates input, builds features, runs the models, applies business rules and renders pages. The model layer is a single serialised artefact containing the failure classifier, failure-type model, anomaly detector and RUL regressor. The data layer is a SQLite database and the CSV dataset used for training.

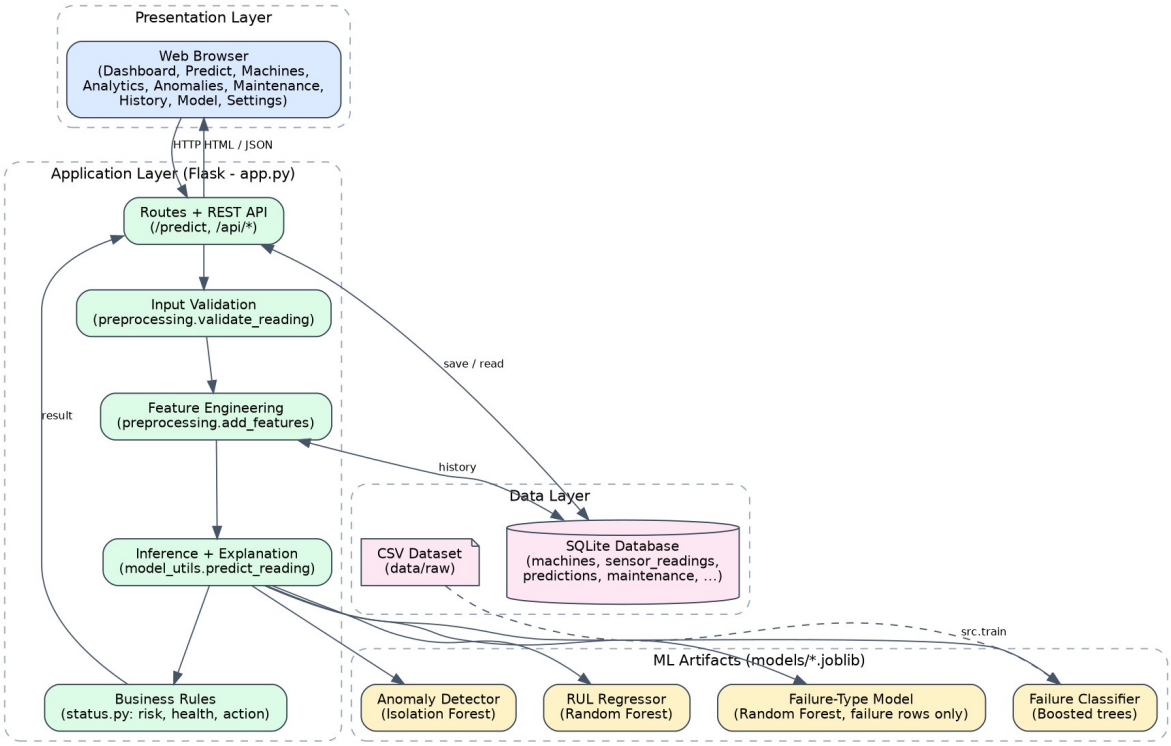


Figure 4.1: Layered system architecture

4.2 Machine Learning Pipeline

The offline training pipeline is run by src/train.py. It loads and cleans the data, builds features, splits it by time, compares four models, evaluates them, trains the three additional models, and saves everything as one joblib artefact together with metrics.json, model_comparison.csv and the figures used in this report.

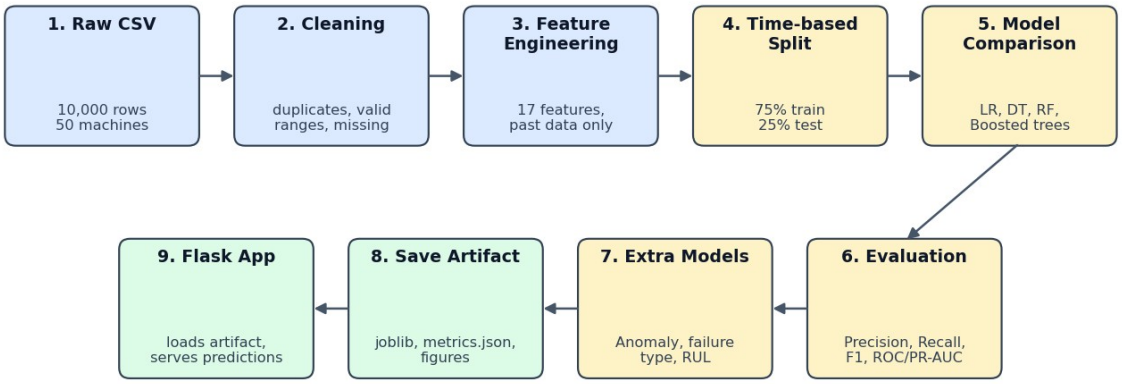


Figure 4.2: Machine learning training pipeline

4.3 Prediction Flow

When a user submits a reading, the application validates it, loads the recent history of that machine from the database, and calls the same add_features() function used in training. The models run on the resulting feature row. An explanation is computed only when the risk is at least MEDIUM, because there is nothing to explain for a healthy machine. Business rules in status.py convert the probability into a risk level, machine health, priority and recommended action, and the result is saved and displayed.

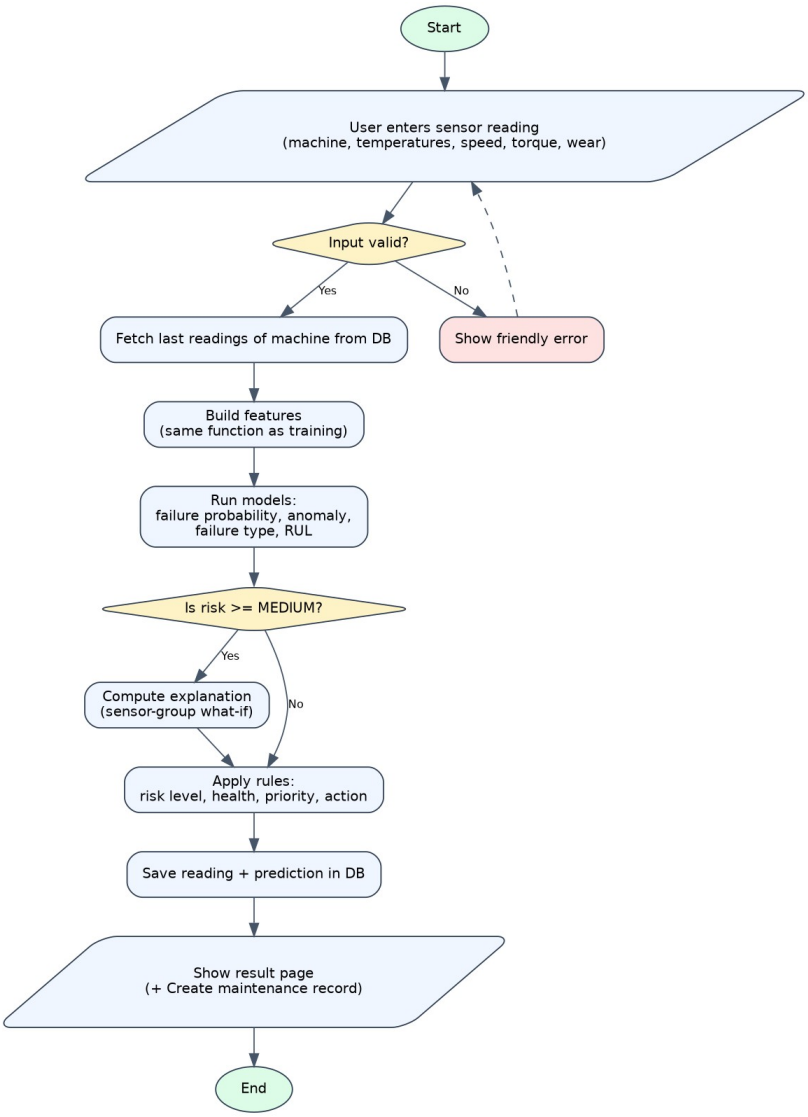


Figure 4.3: Flowchart of a single prediction request

4.4 Database Design

SQLite stores five main tables. The machines table is the parent; sensor readings, failure events, predictions and maintenance records refer to it through machine_id. Indexes on (machine_id, timestamp) keep the history queries fast.

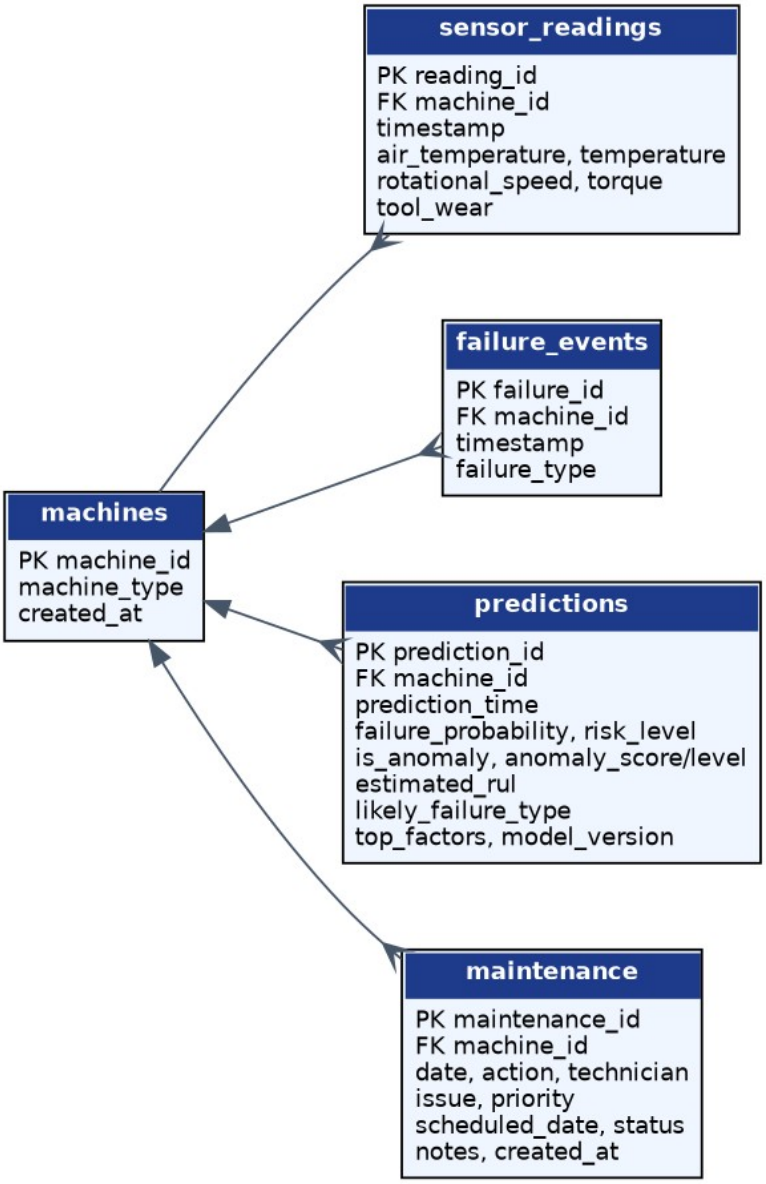


Figure 4.4: Entity-relationship diagram of the database

Table 4.1: Main database tables

Table	Purpose	Key columns
machines	List of machines	machine_id (PK), machine_type
sensor_readings	History of sensor values	machine_id (FK), timestamp, temperatures, speed, torque, tool_wear
failure_events	Past failures	machine_id (FK), timestamp, failure_type
predictions	Every prediction made	failure_probability, risk_level, anomaly, estimated_rul, likely_failure_type, top_factors
maintenance	Maintenance records	machine_id (FK), action, technician, priority, scheduled_date, status
ml_models	Model versions	version, algorithm, metrics

4.5 Module Description

Table 4.2: Source modules

File	Responsibility
config.py	All settings in one place: paths, column names, feature list, thresholds, actions.
src/preprocessing.py	Loading, cleaning, validation, feature engineering, RUL labels, inference-time feature building.
src/train.py	Complete training pipeline, evaluation, plots and artefact saving.
src/model_utils.py	Boosted classifier factory, explanation, single and batch prediction.
src/status.py	Risk, anomaly level, health status, priority and recommended action rules.
src/database.py	SQLite schema, inserts and all queries used by the pages.
src/charts.py	Inline SVG charts (no JavaScript library).
src/data_generator.py	Synthetic dataset generator following the AI4I structure.
src/import_ai4i.py	Converts the real AI4I 2020 CSV to the project format.
app.py	Flask application: nine pages, REST API and error handling.
tests/test_app.py	46 automated tests.

Chapter 5: Implementation

5.1 Dataset

The default dataset (data/raw/predictive_maintenance.csv) contains 10,000 hourly readings from 50 machines between 1 and 9 January 2026. It follows the AI4I 2020 structure, with machine_id and timestamp added so that rolling and trend features can be computed per machine. The overall failure rate is 2.96% (296 failure rows). Because the dataset is synthetic, results on it show that the pipeline works and are not a promise of the same accuracy on a real plant.

Table 5.1: Dataset columns

Column	Meaning	Unit / values
machine_id	Machine identifier	M001 - M050
timestamp	Time of the reading	hourly
machine_type	Quality / product variant	L, M, H
air_temperature	Ambient air temperature	Kelvin (about 300 K = 27 C)
temperature	Process temperature	Kelvin (about 310 K = 37 C)
rotational_speed	Spindle speed	rpm
torque	Torque applied	Nm
tool_wear	Cumulative tool usage	minutes
failure	Target: 0 = healthy, 1 = failure	0 / 1
failure_type	Type of failure	No Failure, Heat Dissipation, Tool Wear, Power, Overstrain, Random

Temperatures are stored in Kelvin, as in the original AI4I dataset. Kelvin is the SI unit of temperature and 0 K is absolute zero, so differences (such as process temperature minus air temperature) have the same value in Kelvin and in degrees Celsius.



Figure 5.1: Class balance: failures are only about 3% of the rows

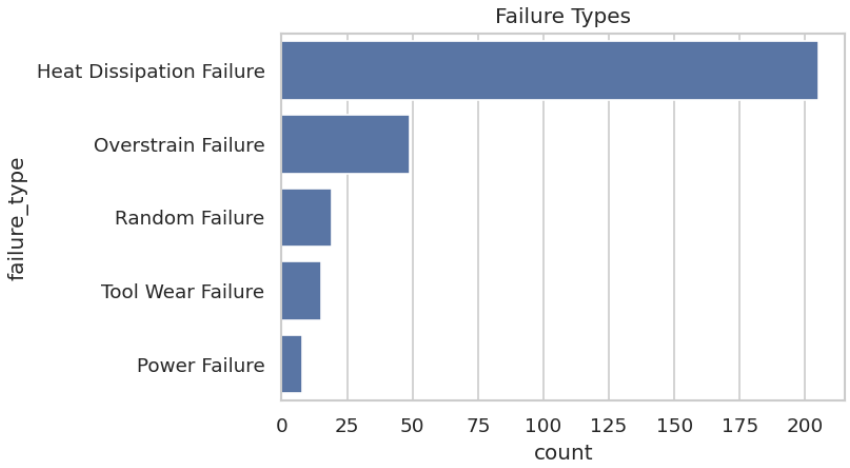


Figure 5.2: Distribution of failure types

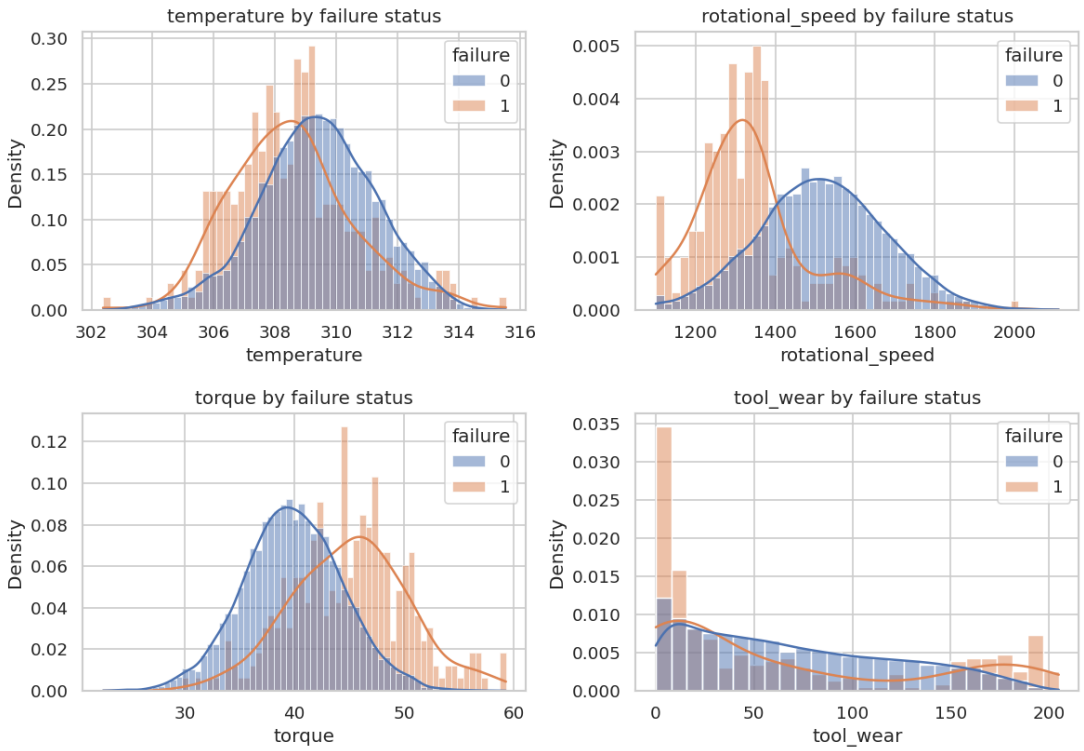


Figure 5.3: Sensor distributions for healthy and failed rows

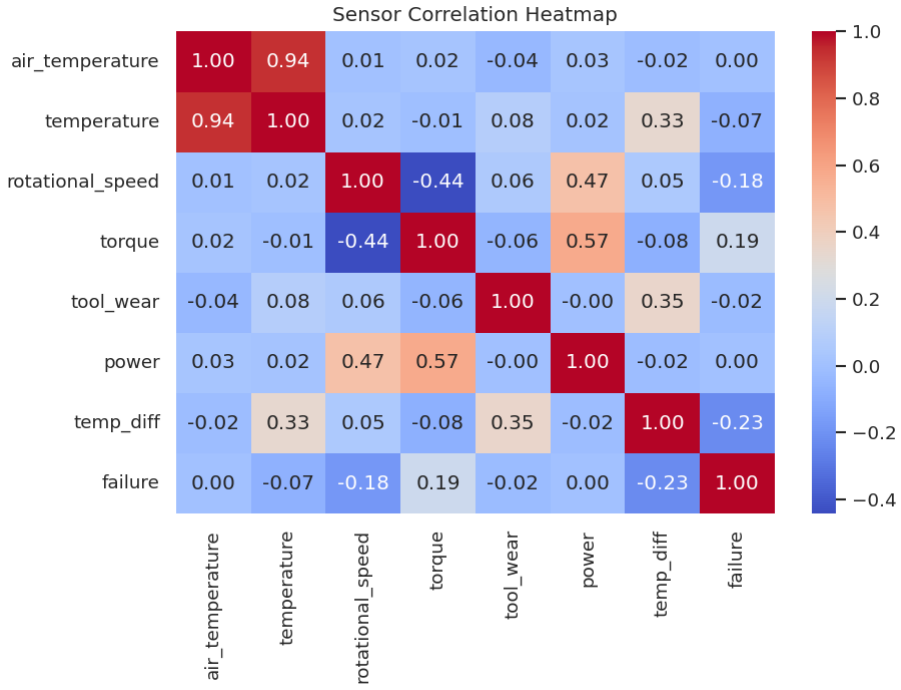


Figure 5.4: Correlation heatmap of the main sensor values

5.2 Data Cleaning

The clean() function removes duplicate rows, converts sensor columns to numbers, marks physically impossible values (for example a negative torque or a temperature outside 250-450

K) as missing, fills gaps with the last known value of the same machine and finally with the median, and sorts the data by machine and time. Unusual but possible values are not deleted, because they may be genuine warning signs.

5.3 Feature Engineering

A single reading rarely shows that a machine is degrading, so 17 features were engineered from the raw values. All rolling and difference features use only past readings of the same machine (pandas rolling windows look backwards), which prevents future information from leaking into the model.

Table 5.2: Engineered features and their reason

Feature	Definition	Why it is useful
temp_diff	process temp - air temp	Shows whether heat is being dissipated properly
power	torque x speed x 2pi / 60	Actual mechanical load on the drive
torque_speed_ratio	torque / speed	Load relative to speed
temp_roll_mean / temp_roll_std	mean / std of last 10 readings	Smoothed level and instability of temperature
temp_delta	current - previous temperature	Sudden jumps
temp_slope	line fit over last 10 readings	Slow heating trend
torque_roll_mean / torque_delta	rolling mean / difference	Load trend
speed_roll_std	std of last 10 speeds	Speed instability
wear_rate	difference in tool wear	How fast the tool is wearing
type_code	L, M, H mapped to 0, 1, 2	Machine class

A dedicated unit test builds a feature from only the first eight rows and from the full history and checks that both give the same value, proving that no future data is used.

5.4 Train / Test Split

A random split would put readings from the future into the training set and give an unrealistically good score. The data is therefore split by time: the first 75% of the timeline (up to 2026-01-07 05:15:00) is used for training (7500 rows) and the last 25% for testing (2500 rows). The model is evaluated only on data that comes after everything it has seen.

5.5 Handling Class Imbalance

Only about 3% of the rows are failures. SMOTE was not used because creating synthetic points between neighbouring readings is risky for time-ordered sensor data. Instead, class weights are used: scale_pos_weight (boosting) equal to the ratio of healthy to failure rows, and

class_weight set to balanced for the other models. Accuracy is not used to choose the model. The models are judged by Precision, Recall, F1, ROC-AUC and PR-AUC.

5.6 Models

5.6.1 Failure Classifier

Four classifiers are compared: Logistic Regression (with feature scaling), a Decision Tree (depth 6), a Random Forest (200 trees) and a gradient-boosted tree model (300 trees, depth 5, learning rate 0.05). The code uses XGBoost when it is installed and falls back to scikit-learn's HistGradientBoostingClassifier otherwise; both belong to the same family of algorithms. The results in this report were produced with the scikit-learn fallback because XGBoost was not installed in the build environment. Installing XGBoost and re-running python -m src.train regenerates all results with XGBoost.

5.6.2 Anomaly Detection

An Isolation Forest (200 trees, contamination 2%) is trained only on healthy training rows. Its score is turned into a LOW / MEDIUM / HIGH anomaly level according to the distance from the model's decision boundary. Anomaly detection is complementary: it can flag unusual behaviour even when the classifier sees low failure risk, for example a faulty sensor or an unseen failure mode.

5.6.3 Failure-Type Model

A Random Forest predicts the failure type. In the first version it was trained on all rows including the majority class "No Failure", which inflated the weighted F1-score. It is now trained and evaluated only on rows where a failure actually occurred (163 training and 133 test failure rows). At prediction time the binary model first estimates the failure risk and the type model then tells which failure type is most likely.

5.6.4 Remaining Useful Life (experimental)

RUL is defined as the number of readings left until the next failure of the same machine, capped at 30. A Random Forest regressor estimates it. Because the data contains only short histories and many machines have no later failure, this output is experimental and is labelled so in the interface.

5.7 Explainability

For each MEDIUM or HIGH prediction the system groups the 17 features into eight sensor groups (tool wear, process-air temperature gap, process temperature, temperature slope, torque, rotational speed, mechanical power, machine type). Each group is replaced by its training median, one at a time, and the drop in the model's raw failure score (log-odds) is measured. A group whose removal lowers the score the most is the main driver. The drops are converted to percentage shares and displayed as bars. Log-odds are used instead of probability

because the probability saturates near 100% and would hide differences. This is a simple group-level what-if analysis and not a full SHAP computation.

5.8 Risk Rules and Recommended Actions

Table 5.3: Risk levels

Failure probability	Risk level	Priority	Recommended action
>= 0.80	HIGH	Urgent	Inspect machine immediately (within 24 hours)
0.50 - 0.79	MEDIUM	Medium	Schedule inspection soon (within 7 days)
< 0.50	LOW	Low	Continue monitoring

The action text is refined by the likely failure type, for example "Inspect cooling system and airflow" for Heat Dissipation Failure or "Inspect or replace the cutting tool" for Tool Wear Failure. Thresholds and actions are defined in config.py so that a plant can tune them.

5.9 Web Application

The Flask application provides nine pages and a REST API. Pages are rendered with Jinja2 templates and charts are generated as inline SVG on the server, so the application needs no JavaScript chart library and works offline. On the first run the application trains the model if it is missing and fills the database with demo history.

Table 5.4: Web pages

Page	URL	Content
Dashboard	/	Summary cards, risk trend, health donut, machines needing attention, recent alerts
Machines	/machines	All machines with health, risk, probability, anomaly and RUL; filters and search
Machine details	/machine/<id>	Status, explanation, RUL, sensor trends, probability history, maintenance records
Predict Health	/predict	Input form and result page with explanation and "Create maintenance record"
Analytics	/analytics	Failure and risk distributions, health trend, expected failure types
Anomalies	/anomalies	Machines with abnormal behaviour and their risk
Maintenance	/maintenance	Create, view and complete maintenance records
Prediction History	/history	Past predictions per machine with probability chart
Model Performance	/model	Model comparison, metrics, confusion matrix and plots

Page	URL	Content
Settings	/settings	Model info, thresholds, database counts and data drift check

Table 5.5: REST API endpoints

Endpoint	Method	Purpose
/api/predict	POST	Predict from a JSON reading
/api/summary	GET	Dashboard summary numbers
/api/machines	GET	List of machines with latest status
/api/machine/<id>	GET	Details of one machine
/api/history/<id>	GET	Prediction history of one machine
/api/maintenance	GET / POST	List or create maintenance records
/api/drift	GET	Simple data drift report
/health	GET	Health check and model status

Chapter 6: Results and Discussion

6.1 Evaluation Setup

All models were trained on the first 75% of the timeline (7500 rows) and evaluated on the last 25% (2500 rows), which contain 133 real failures. A threshold of 0.5 was used for the hard predictions. Results were generated with the scikit-learn HistGradientBoosting model as the primary model.

6.2 Model Comparison

Table 6.1: Model comparison on the test period (synthetic dataset)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.746	0.170	0.970	0.289	0.937	0.491
Decision Tree	0.969	0.649	0.917	0.760	0.934	0.879
Random Forest	0.986	0.971	0.767	0.857	0.977	0.933
HistGradientBoosting	0.989	0.908	0.887	0.897	0.978	0.937

Accuracy is shown only to demonstrate why it is not used for model selection: Logistic Regression has the lowest accuracy but the highest recall, and the Decision Tree and the boosted model have almost the same accuracy but very different precision. The comparison leads to the following observations.

- **Logistic Regression** catches almost all failures (recall 0.970) but with a precision of only 0.170, i.e. most alarms are false. The relation between the sensors and failure is non-linear, which a linear model cannot represent.
- **Decision Tree** has good recall (0.917) but low precision (0.649) and is unstable.
- **Random Forest** has the best precision (0.971) but misses about one in four failures (recall 0.767).
- **Gradient boosting** gives the best balance: F1 0.897, PR-AUC 0.937, with precision 0.908 and recall 0.887. It is therefore the primary model.

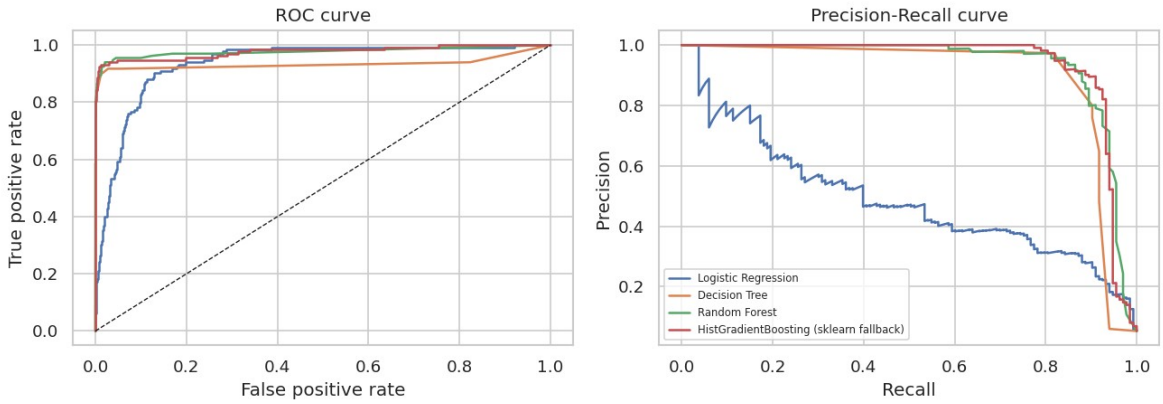


Figure 6.1: ROC and Precision-Recall curves of all models

6.3 Confusion Matrix

For the primary model, of 2367 healthy readings 2355 were classified correctly and 12 raised a false alarm. Of 133 real failures 118 were detected and 15 were missed. In maintenance, a missed failure usually costs more than a false alarm, so the threshold can be lowered to gain recall at the price of more inspections; the notebook 03_model_training shows this trade-off for several thresholds.

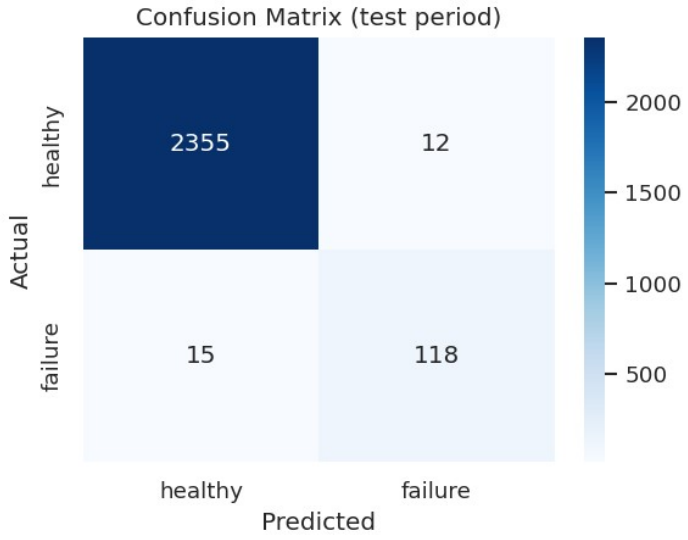


Figure 6.2: Confusion matrix of the primary model on the test period

6.4 Feature Importance

Permutation importance on the test data (drop in PR-AUC when a feature is shuffled) ranks temp diff, rotational speed, tool wear, torque, type code as the five most important features. This agrees with the physical failure modes: the temperature gap relates to heat dissipation failure, the speed and torque to power and overstrain failures, and tool wear to tool wear failure.

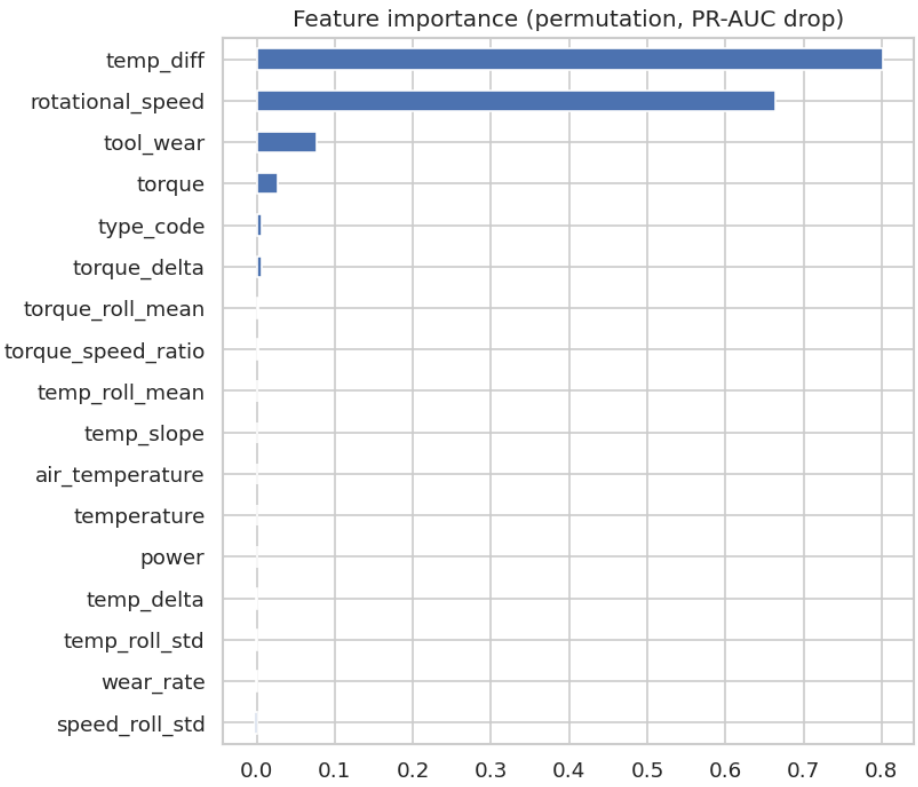


Figure 6.3: Permutation feature importance

6.5 Failure-Type Model

Trained and evaluated on failure rows only, the failure-type Random Forest reaches a weighted F1-score of 0.949 on 133 test failures. This is a more honest number than the earlier version, which was evaluated on all rows. Note that Heat Dissipation Failure is the dominant failure type in this dataset (205 of 296 failure rows), so rare types such as Power or Tool Wear failure have very few examples and their individual scores are less reliable.

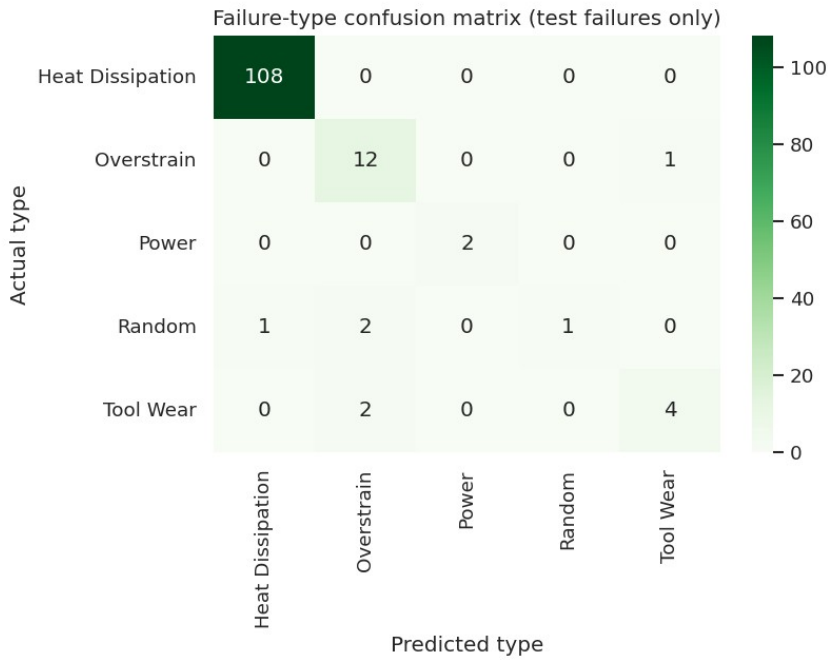


Figure 6.4: Failure-type confusion matrix (failure rows only)

6.6 Anomaly Detection and RUL

Table 6.2: Anomaly detector and RUL regressor (synthetic dataset)

Component	Result
Isolation Forest	131 of 2500 test readings flagged; overlap with real failures: precision 0.481, recall 0.474
RUL regressor	MAE 7.75 readings, RMSE 9.93, R2 0.253; predicting the training mean gives MAE 10.22

The anomaly detector is unsupervised and is not meant to predict failures; its overlap with real failures shows that about half of the failures are also visible as unusual behaviour, while it additionally flags other unusual readings. The RUL model improves on the naive mean baseline, but with an R2 of only about 0.25 it is a rough indicator, so it is marked as experimental in the application.

6.7 Comparison with the Real AI4I 2020 Dataset

To check that the pipeline is not tuned only to the synthetic data, the real AI4I 2020 file was converted with src/import_ai4i.py and the same training script was run. The original dataset has no machine_id or timestamp, so rows are grouped into pseudo-machines of 200 consecutive rows with hourly timestamps. Rolling and trend features are therefore much less meaningful on this dataset, and this is expected to lower the scores.

Table 6.3: Primary model on synthetic data vs real AI4I 2020 data

Metric	Synthetic dataset	Real AI4I 2020
Precision	0.908	0.752
Recall	0.887	0.840
F1-score	0.897	0.794
ROC-AUC	0.978	0.984
PR-AUC	0.937	0.886
Failure-type weighted F1	0.949	0.968
Failure rate in data	2.96%	3.39%

On the real data the boosted model still performs best among the four models (F1 0.794 against 0.764 for Random Forest, 0.557 for the Decision Tree and 0.292 for Logistic Regression) and its ROC-AUC is 0.984. The lower F1-score compared to the synthetic data is expected because real records are noisier and the time-based features carry less information. The result also shows that the pipeline runs on a real, externally published dataset without code changes.

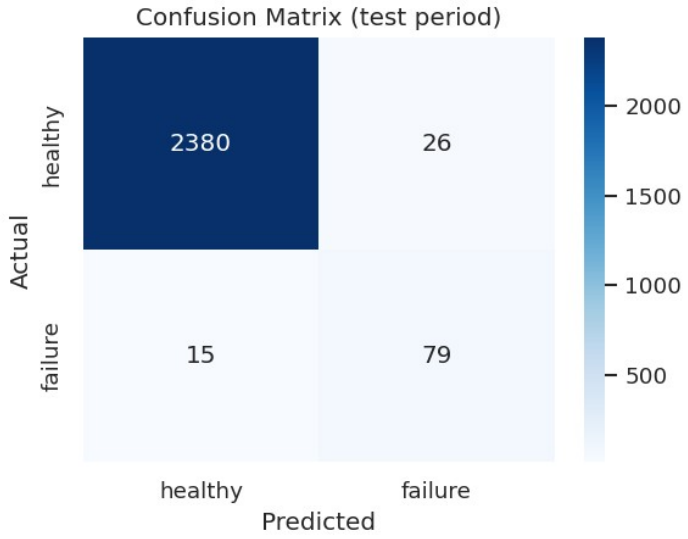


Figure 6.5: Confusion matrix on the real AI4I 2020 data

6.8 Discussion

- Time-based validation and class weights give results that can be trusted more than a random split with accuracy.
- The best model is chosen by the balance of precision and recall, not by one number; the balance can be moved by changing the threshold.
- The failure-type result was corrected by evaluating it on failure rows only, which lowers inflated numbers but is more honest.

- The real-data experiment shows lower but still good scores and highlights how much the time-series features depend on proper machine histories.

Chapter 7: Testing

Testing is done with Python's unittest framework (python -m unittest discover -s tests -v). The tests use a temporary database so the real database is not changed. All 46 tests pass.

Table 7.1: Automated tests

Test group	Tests	What is checked
PreprocessingTests	4	Cleaning removes duplicates and fills gaps; all model features exist; rolling features use only past data (leakage test); default air temperature
StatusRuleTests	4	Risk thresholds, anomaly level, health status, priority and recommended action
ChartTests	3	SVG charts for empty/tiny data, flat lines, escaping of labels
AppTests	30	Pages load, prediction output fields, normal vs risky sample, missing or non-numeric input, history saving, maintenance create/complete, safe redirect, HTML escaping of notes, API responses, missing-model error page, sidebar
Total	46	All passed

Besides the automated tests, the application was run and every page was opened in a browser; the screenshots in Chapter 8 were captured from this run.

Table 7.2: Sample manual test cases

Input	Expected	Observed
Normal reading (300 K, 309.6 K, 1500 rpm, 40 Nm, 30 min)	LOW risk, no explanation	LOW risk, "Machine looks normal"
Risky reading (302 K, 313.5 K, 1250 rpm, 62 Nm, 220 min)	HIGH risk with explanation	HIGH risk (99.7%), Overstrain Failure, tool wear and torque as main factors
Temperature left empty	Validation error message	Friendly message: "Process temperature is required."
Text in a numeric field	Validation error message	Friendly message, no crash
Torque far outside typical range	Warning shown	Warning "torque value is far outside the usual range"

Chapter 8: User Interface Screenshots

The following screenshots were taken from the running application. Long pages are cropped to the visible top part.

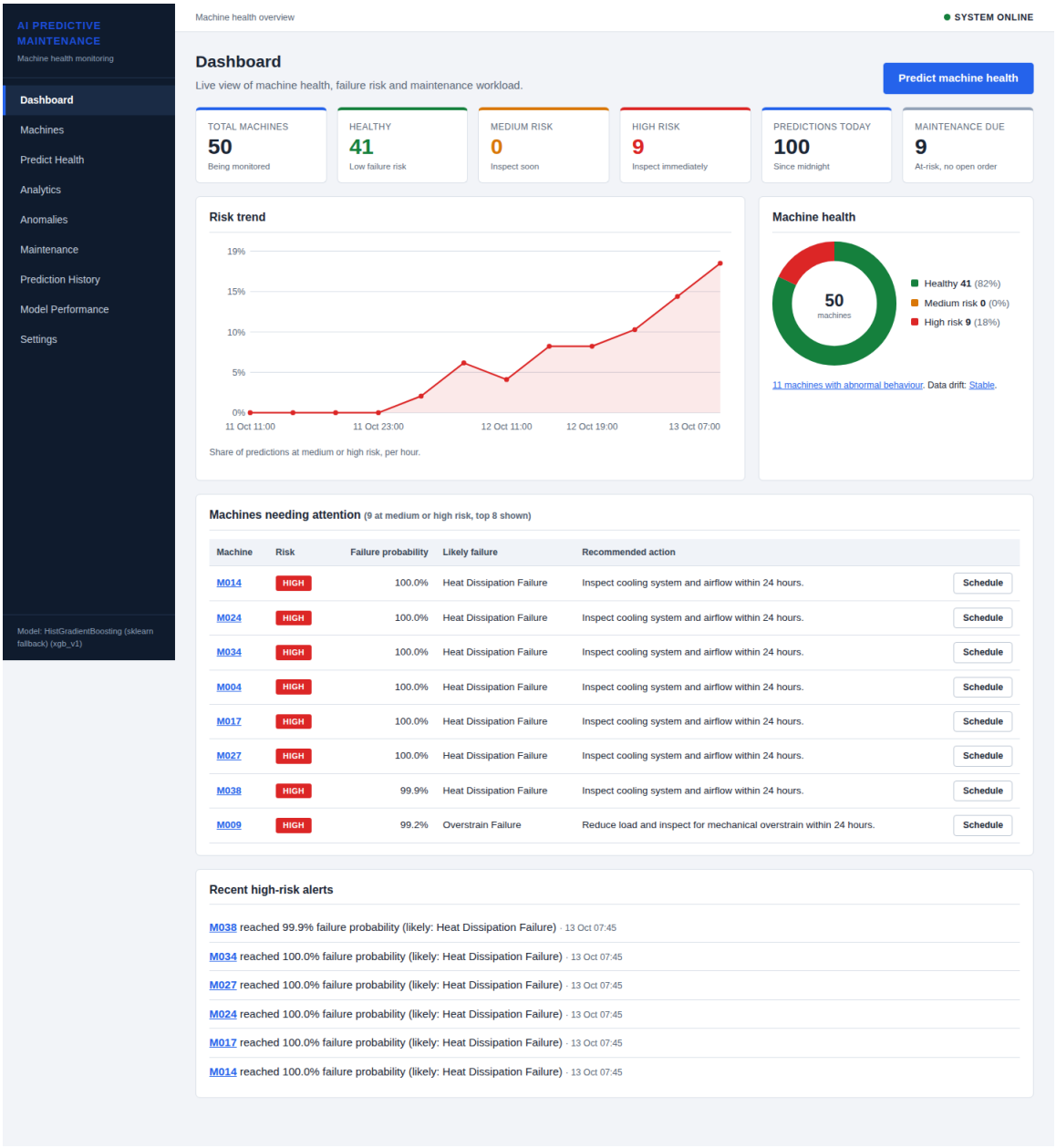


Figure 8.1: Dashboard with summary cards, risk trend and machines needing attention

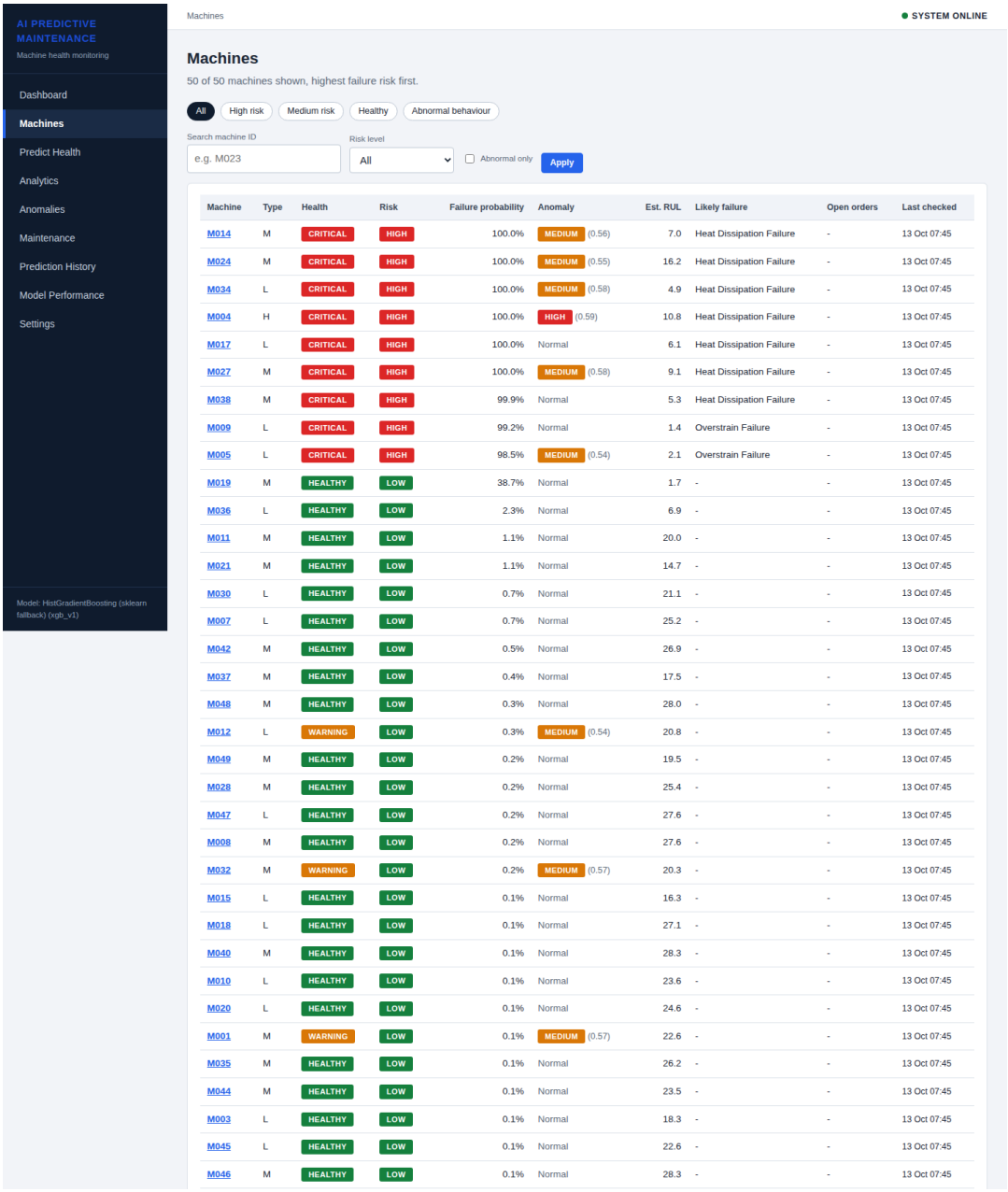


Figure 8.2: Machines page with health, risk, anomaly and RUL columns

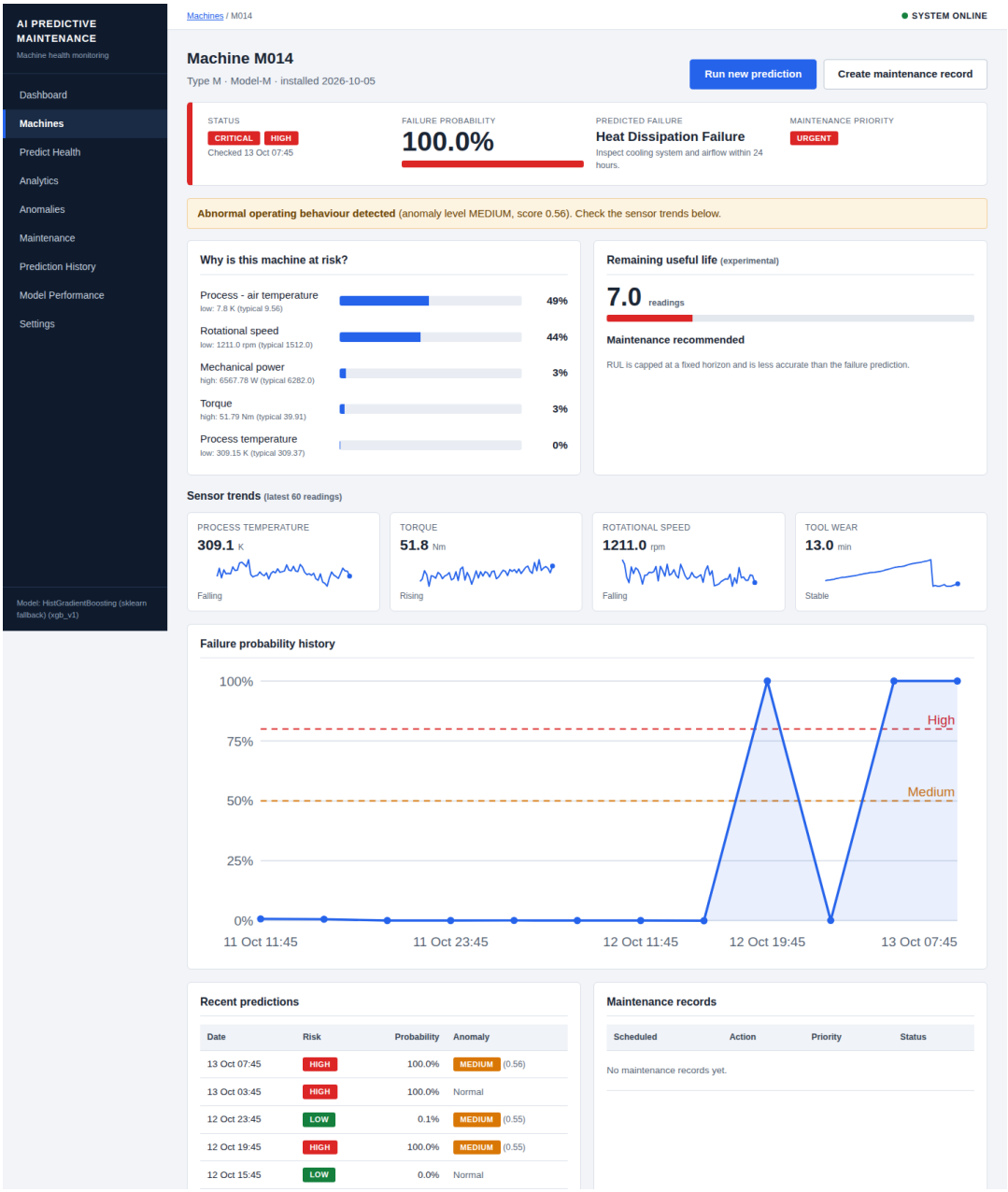


Figure 8.3: Machine detail page with explanation, sensor trends and history

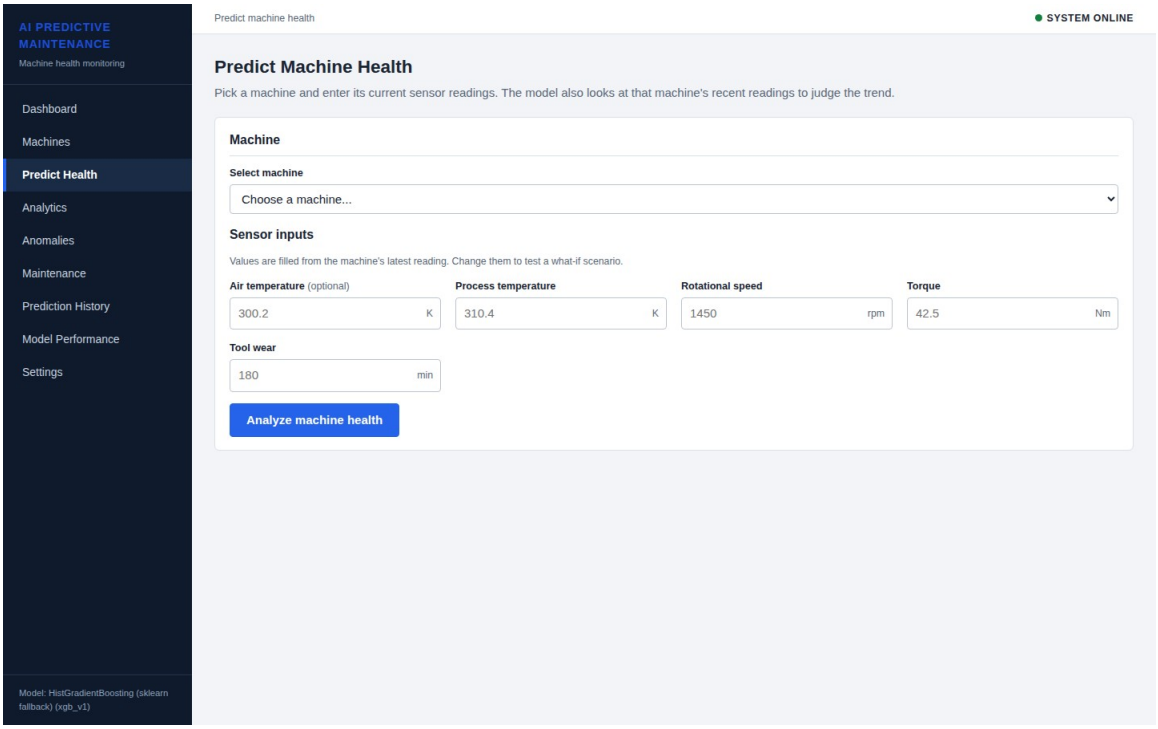


Figure 8.4: Predict Health input form

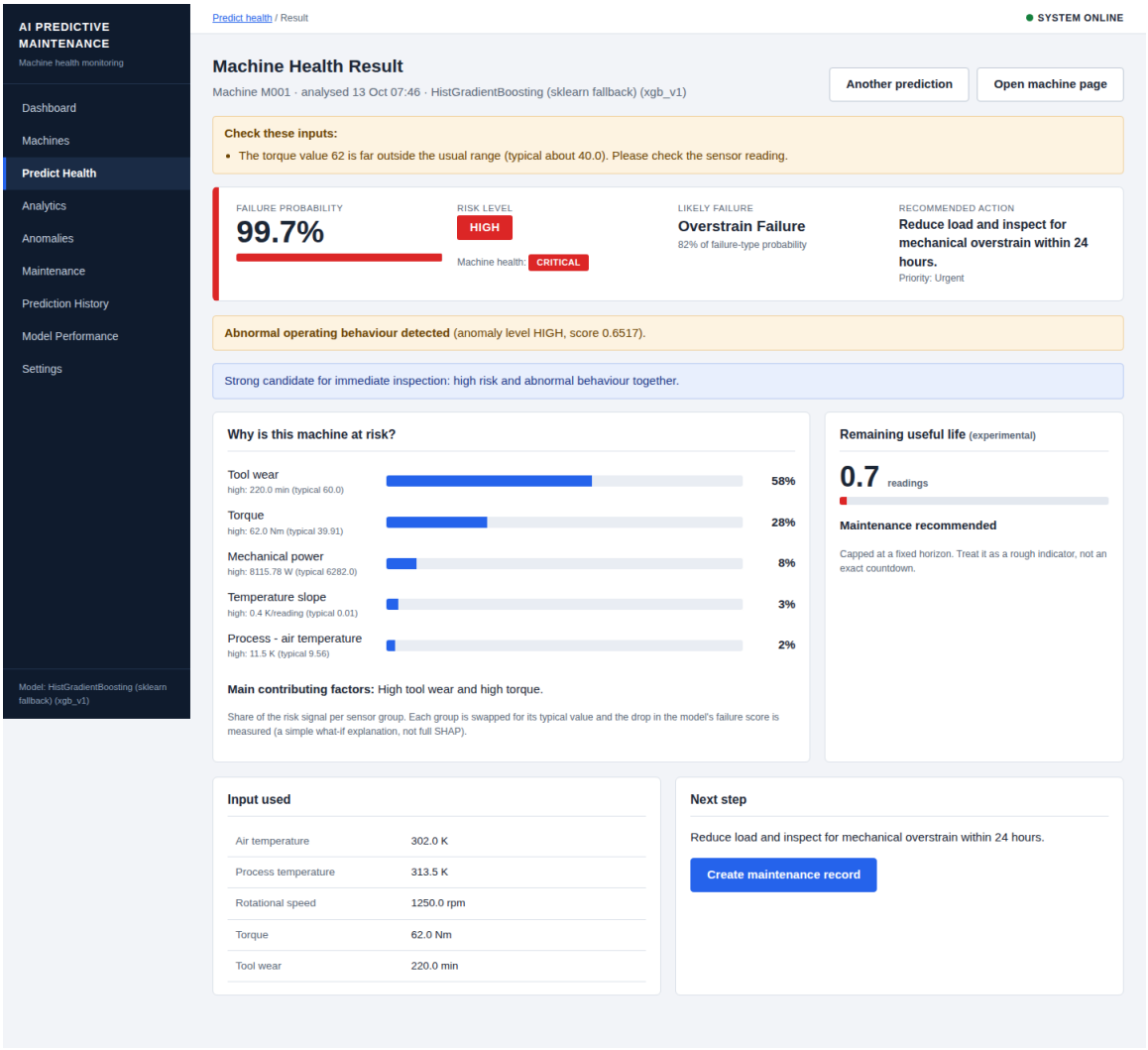


Figure 8.5: Prediction result with risk, likely failure, explanation and next step

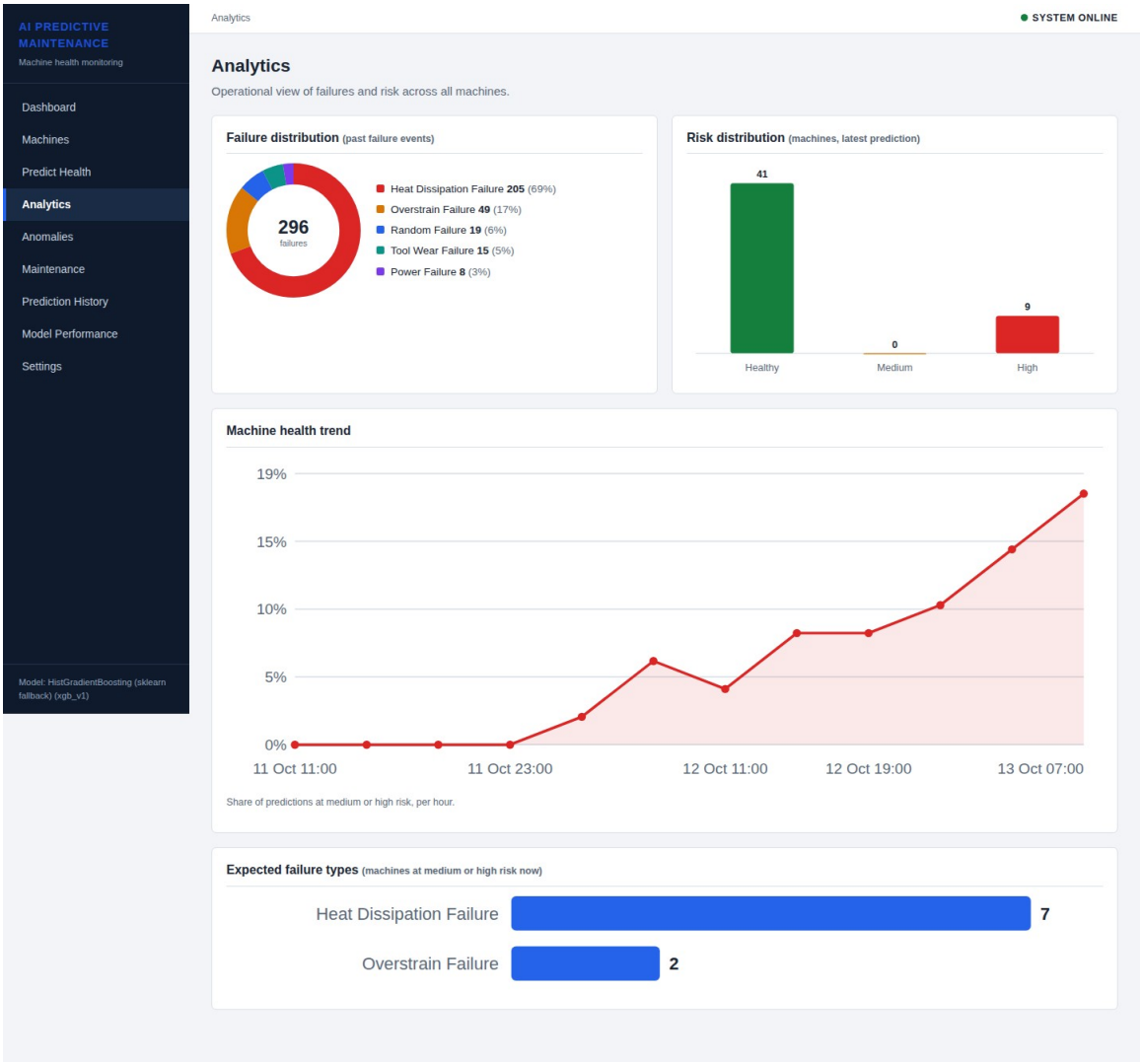


Figure 8.6: Analytics page

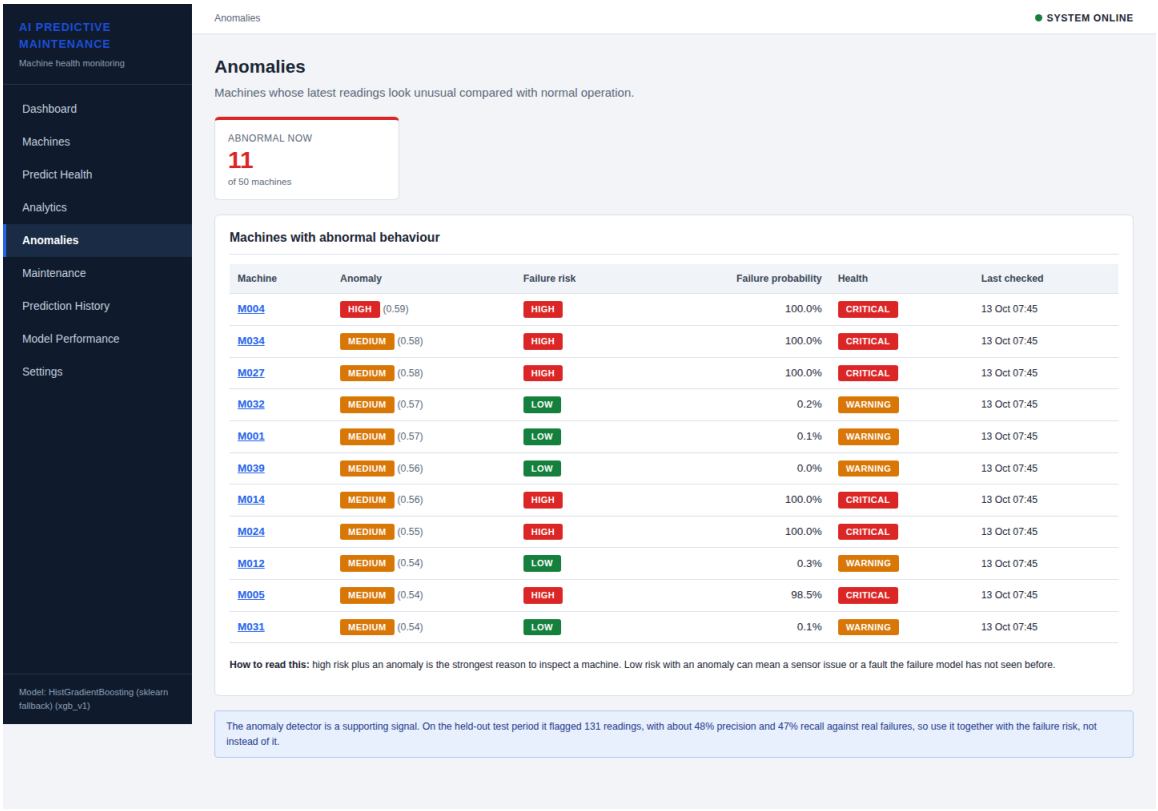


Figure 8.7: Anomalies page

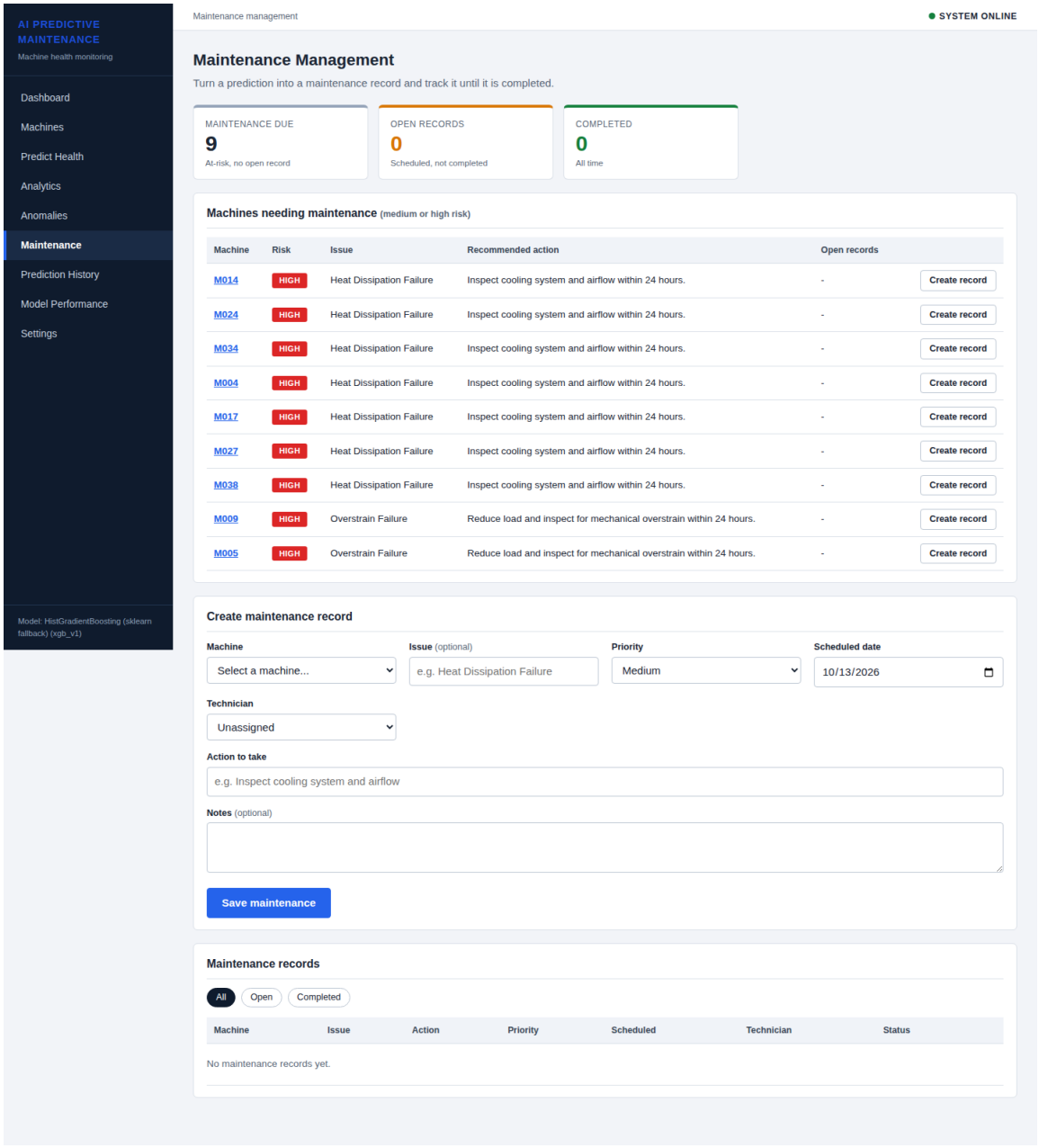


Figure 8.8: Maintenance page with records and creation form

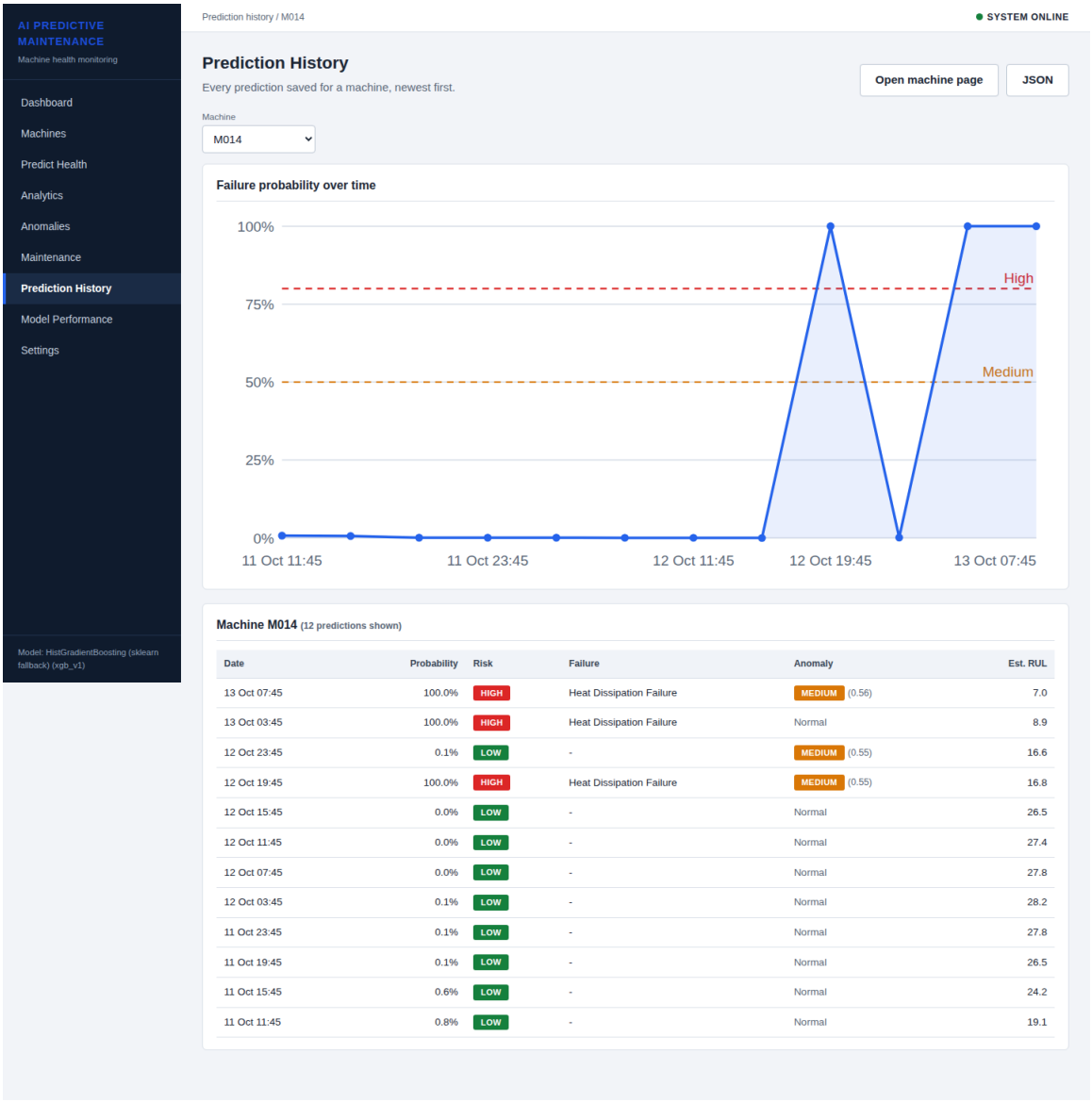


Figure 8.9: Prediction History page

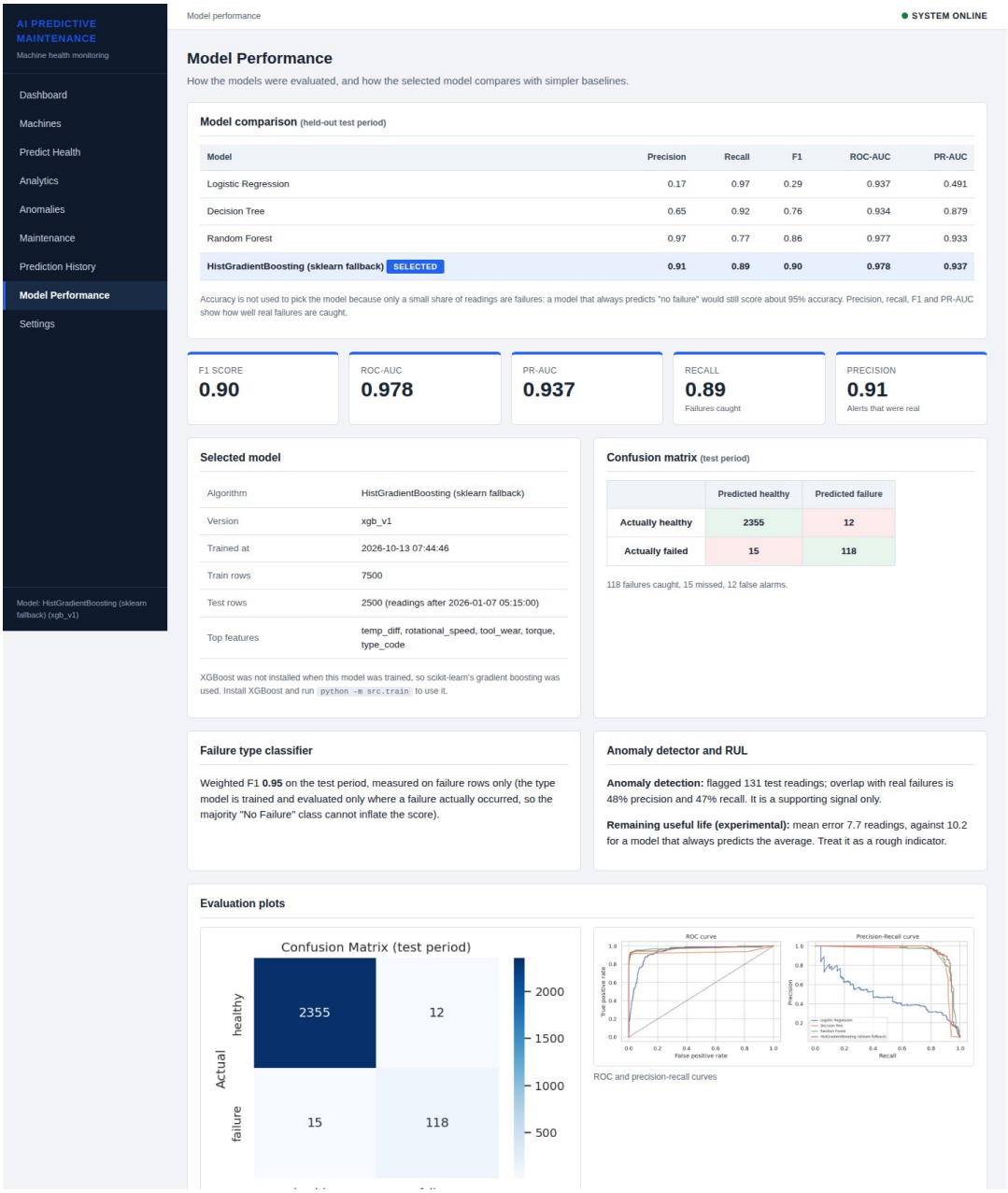


Figure 8.10: Model Performance page

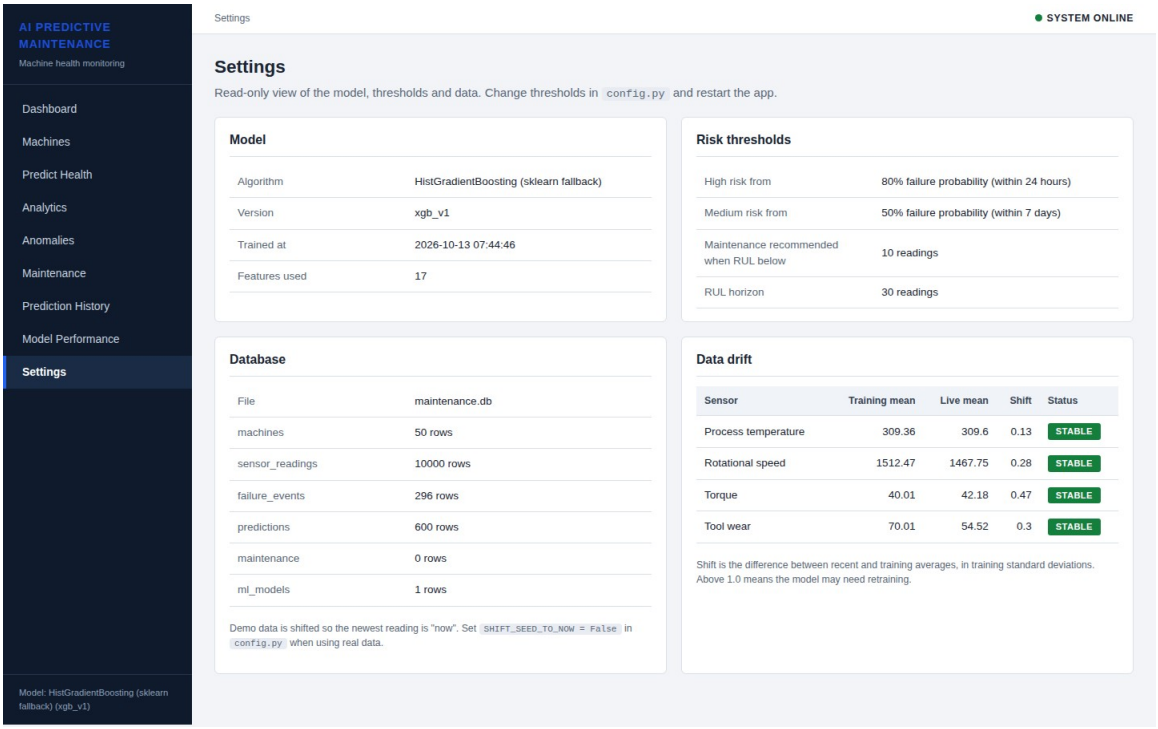


Figure 8.11: Settings page with model information, thresholds and drift check

Chapter 9: Limitations

- **Synthetic dataset:** the default data is generated and follows the AI4I 2020 structure; results on a real plant will differ. Real AI4I 2020 results are reported for comparison.
- **Short histories:** each machine has about 200 readings over nine days, which limits long-term degradation learning and RUL accuracy.
- **RUL is experimental:** R2 is about 0.25; it is a rough indicator only.
- **Rare failure types:** some types (Power, Tool Wear, Random) have very few examples, so type-level scores are uncertain.
- **Explainability is approximate:** sensor-group what-if analysis is not SHAP and does not model feature interactions exactly.
- **No authentication / CSRF protection:** the application is a single-user demo. The SECRET_KEY is generated randomly at start-up unless it is set through an environment variable, which must be done in any real deployment.
- **No live sensor connection:** readings are entered manually or through the API.
- **XGBoost not used in the reported run:** the scikit-learn gradient boosting fallback was used because XGBoost was not installed in the build environment.
- **AI4I rows are not a time series:** on the real dataset the machine/timestamp structure is artificial, which weakens rolling features.

Chapter 10: Conclusion and Future Scope

10.1 Conclusion

This project built a complete predictive maintenance system and not only a model. The machine learning part follows good practice for imbalanced time-ordered data: a time-based split, no leakage, class weights instead of SMOTE, and evaluation with Precision, Recall, F1, ROC-AUC and PR-AUC. The gradient-boosted model achieved a precision of 0.908, recall of 0.887 and F1-score of 0.897 on the unseen test period and gave the best balance among the four models compared. Additional components for anomaly detection, failure-type prediction, RUL estimation and a lightweight explanation make the output useful to a maintenance team. The Flask dashboard, database, REST API and maintenance workflow turn a prediction into an action, and 46 automated tests protect the behaviour of the system. The experiment on the real AI4I 2020 dataset (F1 0.794, ROC-AUC 0.984) shows that the pipeline generalises to external data, with the expected drop caused by noisier records and missing time-series structure.

10.2 Future Scope

- Train and validate on real plant data with proper machine histories, and connect live sensors through MQTT or OPC-UA.
- Use deep sequence models (LSTM, temporal convolution) for RUL when long run-to-failure histories are available.
- Add SHAP or other model-agnostic explanation methods and compare them with the group-level what-if method.
- Tune the decision threshold from the real cost of a missed failure versus a false alarm, and add probability calibration.
- Add user login, role-based access, CSRF protection and a production WSGI server for deployment.
- Automate periodic retraining and drift-triggered alerts, and send notifications by e-mail or SMS.
- Package the application with Docker for easy deployment.

References

1. S. Matzka, "Explainable Artificial Intelligence for Predictive Maintenance Applications," in Proc. 3rd International Conference on Artificial Intelligence for Industries (AI4I), 2020, pp. 69-74. (AI4I 2020 Predictive Maintenance Dataset.)
2. T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, R. P. Francisco, J. P. Basto and S. G. S. Alcala, "A systematic literature review of machine learning methods applied to predictive maintenance," Computers & Industrial Engineering, vol. 137, 106024, 2019.
3. A. Saxena, K. Goebel, D. Simon and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in Proc. International Conference on Prognostics and Health Management (PHM), 2008.
4. L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.
5. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785-794.
6. F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE International Conference on Data Mining (ICDM), 2008, pp. 413-422.
7. N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321-357, 2002.
8. T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," PLoS ONE, vol. 10, no. 3, e0118432, 2015.
9. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
10. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
11. Flask Documentation, Pallets Projects. https://flask.palletsprojects.com

Note: please cross-check bibliographic details (page numbers, volume) against the original sources before final submission.

Appendix A: Project Structure and How to Run

A.1 Folder Structure

```
predictive-maintenance/
|-- app.py                  Flask application (pages, API)
|-- config.py               settings: paths, thresholds, features
|-- requirements.txt
|-- data/raw/               predictive_maintenance.csv (synthetic)
|-- data/external/          ai4i2020.csv (real AI4I 2020 data)
|-- models/                 predictive_maintenance.joblib (trained)
|-- notebooks/              01_eda, 02_feature_engineering,
|                           03_model_training, 04_real_ai4i_comparison
|-- reports/                metrics.json, model_comparison.csv,
|                           figures/, ai4i_real/
|-- src/                    preprocessing, train, model_utils, status,
|                           database, charts, data_generator, import_ai4i
|-- templates/, static/     HTML pages and CSS
|-- tests/test_app.py       46 automated tests
|-- docs/                   VIVA_NOTES.md, architecture diagrams, report
```

A.2 Commands

```
pip install -r requirements.txt
python -m src.train                # train models, create plots and metrics
python app.py                      # start web app at http://127.0.0.1:5000
python -m unittest discover -s tests -v   # run the 46 tests
python -m src.import_ai4i data/external/ai4i2020.csv   # use real AI4I data
```

A.3 Example API Call

```
POST /api/predict
{"machine_id": "M001", "machine_type": "L", "air_temperature": 302,
 "temperature": 313.5, "rotational_speed": 1250, "torque": 62, "tool_wear":
220}
```

The response contains failure_probability, risk_level, recommended_action, priority, likely_failure_type, is_anomaly, anomaly_level, estimated_rul and top_factors.